PLANORA AI

2-Hour Hackathon MVP Build Plan

AI-powered family celebration planner
Plan Smarter. Celebrate Better.

Goal: Build and deploy a small but working web prototype in 120 minutes. Do not attempt the full PRD. Focus on one complete demo journey: create a birthday event, generate a plan, change the budget/guest count, and show an updated plan.

1. What to build (strict MVP scope)

Build now	Skip for this 2-hour version
• Single event creation form • Gemini-generated event plan • Budget allocation display • Simple task checklist • Change budget and guest count • AI-generated revised plan • Save event/plan in Supabase if time allows	• Full authentication flows • Vendor marketplace or live discovery • Guest RSVP system • Notifications and calendar sync • Multi-agent architecture • Complex Gantt charts • Multiple event collaboration

2. Recommended stack

Frontend + API: Next.js App Router + TypeScript
UI: Tailwind CSS (use existing components; avoid spending time on design systems)
AI: Gemini API through a server-side Route Handler
Persistence: Supabase PostgreSQL only if setup is already familiar; otherwise use local in-memory/demo state for the first working prototype and add persistence if time remains.
Deployment: Vercel. Keep Gemini keys server-side in environment variables.

3. Exact 120-minute schedule

Time	Task	Done when
0–10 min	Create Next.js app, install dependencies, create GitHub repo, add Gemini key to .env.local.	App runs locally; secret is not committed.
10–30 min	Build one page: event form with event type, date, city, guests, budget, food/theme preferences.	Form accepts and validates inputs.
30–50 min	Create POST /api/plan route; call Gemini with a strict JSON prompt; validate response.	Clicking Generate returns a structured plan.
50–70 min	Render event summary, budget categories, tasks and assumptions. Add loading/error states.	Initial plan is readable and usable.

Time	Task	Done when
70–95 min	Add change controls for new budget and guest count; create POST /api/replan using old plan + changes.	Updated plan and trade-offs appear.
95–105 min	Add lightweight persistence/version history (Supabase if ready; otherwise browser state for demo).	Original and revised plans can be compared.
105–115 min	Test the complete demo, fix critical bugs, deploy to Vercel.	Public URL works.
115–120 min	Record a short walkthrough and verify GitHub, deployment and environment settings.	Submission links are ready.

4. Minimum screen design

Single-page dashboard is enough. Avoid building separate pages unless the core workflow is complete.

Header: Planora AI logo + “New event”
Left/top section: Event input form
Main section: Generated plan with event summary and budget cards
Budget section: Category names, amounts and total
Tasks section: Checklist with due dates
Change panel: New budget + new guest count + “Replan event” button
History panel: Original plan vs revised plan

5. Gemini workflow and prompt

Keep it to two AI calls:
1. Generate the initial plan.
2. Replan using the previous plan and updated constraints.

```
You are Planora AI, a practical family event planning assistant.

Create an actionable plan from the user's event details.
Return JSON only, matching this shape:
{
"summary": "string",
"budget_allocations": [
{"category":"Venue","amount":0,"reason":"string"}
],
"tasks": [
{"title":"string","due_date":"YYYY-MM-DD","priority":"high|medium|low"}
],
"assumptions": ["string"],
"trade_offs": ["string"]
}

Rules:
- Use the user's budget and currency.
- Budget amounts must be non-negative and must not exceed the total budget.
- Do not invent vendor names, quotations or availability.
- Mark estimates as planning estimates, not confirmed prices.
- If information is missing, state assumptions.
- Generate practical tasks before the event date.
```

Replanning prompt

```
You are Planora AI's change-impact assistant.
Compare the previous plan with the updated requirements.
Identify:
- what changed
- affected budget categories
- affected tasks
- practical trade-offs
- revised budget allocations
- revised tasks
Return the same JSON structure as the initial plan.
Never claim vendor availability or invent quotations.
Keep the revised allocations within the new budget.
If the constraints cannot all be satisfied, explain the conflict.
```

6. Minimal API design

Method + route	Purpose
POST /api/plan	Accept event requirements and return a structured initial plan.
POST /api/replan	Accept previous plan + updated budget/guest count and return revised plan.

7. Suggested project structure

```
src/
  app/
    page.tsx
    api/
      plan/route.ts
      replan/route.ts
  components/
    EventForm.tsx
    PlanResults.tsx
    BudgetBreakdown.tsx
    TaskList.tsx
    ReplanPanel.tsx
  lib/
    gemini.ts
    schemas.ts
    budget.ts
  types/
    plan.ts
.env.local
.env.example
README.md
```

8. Budget and replanning rules

• Treat the backend as the source of truth for arithmetic.
• Calculate the revised total budget in TypeScript, not by trusting Gemini.
• Validate that every allocation is non-negative and the sum does not exceed the new budget.
• If the guest count rises, flag catering and venue capacity for review.
• If the event date changes, recalculate task deadlines and flag vendor reconfirmation.
• Show a preview; do not overwrite the accepted plan until the user approves it.

9. Test checklist

■ Create a birthday event with ■50,000 and 100 guests.
■ Generate and display the initial plan.
■ Confirm allocation totals do not exceed ■50,000.
■ Change budget to ■40,000 and guests to 130.
■ Confirm the revised plan explains affected categories and trade-offs.
■ Confirm invalid or malformed AI output produces a useful error.
■ Confirm API key is not present in browser code or Git history.
■ Open the deployed URL and repeat the demo.

10. Deployment and submission

1. Push code to GitHub.
2. Import repository into Vercel.
3. Add GEMINI_API_KEY and any other required server environment variables.
4. Deploy and test the production URL.
5. Include a README with setup instructions, stack, AI workflow and screenshots.
6. Record a 2–3 minute video showing create → generate → change constraints → replan.

11. Demo scenario for judges

Initial input: Birthday celebration in Mumbai, 100 guests, ■50,000 budget, vegetarian food, elegant theme.
Show: AI-generated budget, tasks and assumptions.
Change: Budget drops to ■40,000 and guest count rises to 130.
Prove: Planora AI identifies catering/venue impact, proposes trade-offs, validates the new budget and presents a revised plan. Show the old plan remains available.

12. Final scope rule

If time is running out, prioritize a working initial plan and a working replan flow over authentication, vendor management, guest RSVP, notifications or elaborate UI. A single reliable end-to-end journey is the strongest 2-hour MVP.